

ДИСЦИПЛИНА	Алгоритмы и структуры данных
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Текущий контроль
ПРЕПОДАВАТЕЛЬ	Преснецова Виктория Юрьевна, Яковлев Михаил Сергеевич, Дворецкий Артур Геннадьевич, Гиматдинов Дамир Маратович
СЕМЕСТР	2 семестр, 2024-2025 гг.

Рабочая тетрадь 4.

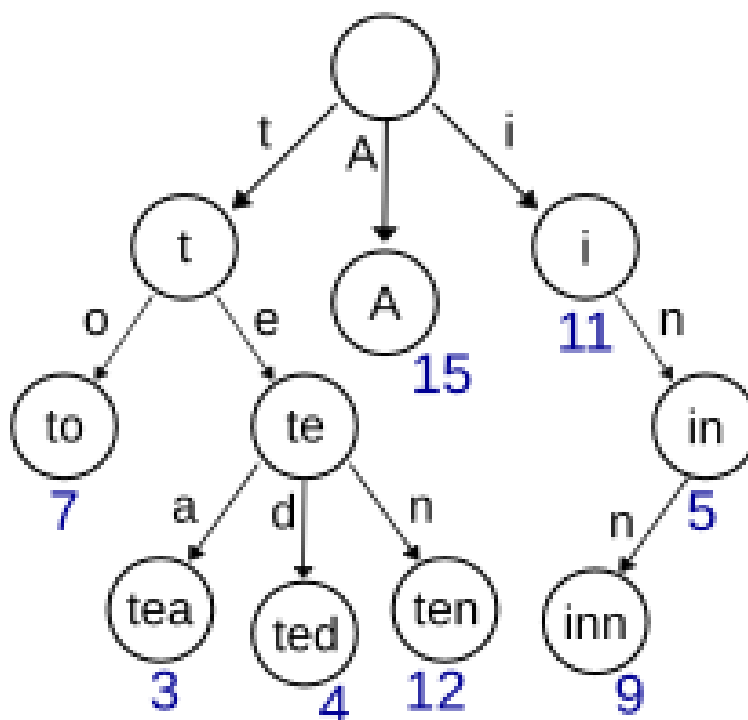
Префиксное дерево. Дерево Хаффмана. Примеры на жадные алгоритмы

Требования
1. Код должен быть оптимизирован для производительности и использования ресурсов. 2. Необходимо избегать избыточных вычислений и памяти. 3. Комментарии должны объяснять сложные участки кода и логику работы программы.

1. Префиксное дерево

Теоретический материал
Префиксное дерево (бор, trie) - это структура данных, используемая для хранения динамических множеств или ассоциативных массивов, где ключами являются строки. Оно представляет собой корневое дерево, в котором каждая вершина соответствует символу строки. Некоторые узлы префиксного дерева выделены (на рисунке они подписаны цифрами) и считается, что префиксное дерево содержит данную строку-ключ тогда и только тогда, когда эту строку можно прочесть на пути из корня до

некоторого (единственного для этой строки) выделенного узла. В некоторых приложениях удобно считать все узлы дерева выделенными.



Основные свойства:

1. Иерархическая структура - строки хранятся в виде пути от корня к листу.
2. Общий префикс - если несколько строк имеют общий префикс, они разделяют начальную часть пути.
3. Операции поиска, вставки и удаления выполняются за $O(L)$, где L - длина строки.

Основные операции:

1. Вставка (insert): добавление слова в дерево.
2. Поиск (search): проверка наличия слова.
3. Проверка префикса (startsWith): проверка существования строк, начинающихся с данного префикса.

Пример 1

Задача:

✘ Реализуйте поиск в префиксном дереве

Решение:

```

1
2 #include <iostream>
3 using namespace std;
4
5 struct TrieNode {
6     TrieNode* children[26];
7     bool isEndOfWord;
8
9     TrieNode() : isEndOfWord(false) {
10         for (int i = 0; i < 26; i++) {
11             children[i] = nullptr;
12         }
13     }
14 };
15 TrieNode* root = new TrieNode();
16
17 void insert(const string& word) {
18     TrieNode* node = root;
19     for (char ch : word) {
20         int index = ch - 'a';
21         if (node->children[index] == nullptr) {
22             node->children[index] = new TrieNode();
23         }
24         node = node->children[index];
25     }
26     node->isEndOfWord = true;
27 }
28
29 bool search(const string& word) {
30     TrieNode* node = root;
31     for (char ch : word) {
32         int index = ch - 'a';
33         if (node->children[index] == nullptr) {
34             return false;
35         }
36         node = node->children[index];
37     }
38     return node->isEndOfWord;
39 }
40
41 int main() {
42     string word;
43     insert("hello");
44     insert("world");
45     insert("heaven");
46     insert("heavy");
47
48     cout << "Введите слово:" << endl;
49     cin >> word;
50
51     cout << "Слово " << word << " ";
52     if (search(word)) {
53         cout << "найдено!" << endl;
54     } else {
55         cout << "не найдено!" << endl;
56     }
57
58
59     return 0;
60 }
61

```

Ответ:

Введите слово:
hi
Слово hi найдено!

Задание 1 (1.5 балла)

Задача:

Реализуйте функцию, которая находит количество слов в префиксном дереве, начинающихся с заданного префикса.

Решение:

Ответ:

Задание 2 (1.5 балла)

Задача:

Реализуйте операцию удаления слова из trie и проверьте, осталось ли оно после удаления.

Решение:

Ответ:

Задание 3*

Задача:

Вам дан список слов. Найдите слово, у которого наибольшее количество других слов списка являются его префиксами.

Вход:

a
ab
abc
abcd
abcdef
bcd

Выход:

abcd (Префиксы: a, ab, abc, abcd)

Решение:	
Ответ:	

2. Жадные алгоритмы. Дерево Хаффмана

Теоретический материал
Жадные алгоритмы - это алгоритмические методы, которые принимают локально оптимальные решения на каждом шаге с надеждой, что они приведут к глобально оптимальному решению. Основной принцип жадных алгоритмов заключается в построении решения поэтапно, где на каждом шаге выбирается лучший возможный вариант. При этом выбор текущего шага никак не изменяется в дальнейшем, что отличает жадные алгоритмы от других подходов, таких как динамическое программирование или метод перебора с откатом. Такой подход оказывается эффективным, если задача обладает свойствами жадного выбора и оптимальной подструктуры. Жадный выбор означает, что выбор локально наилучшего решения на каждом шаге приведёт к оптимальному глобальному решению. Оптимальная подструктура подразумевает, что любое оптимальное решение задачи может быть получено из оптимальных решений её подзадач. Применение жадных алгоритмов обосновано в тех случаях, когда можно доказать, что жадный выбор всегда приводит к глобально оптимальному решению. Однако, если такой гарантии нет, жадные алгоритмы могут давать лишь приближённые результаты. Это отличает их от динамического программирования, которое строит решение путём комбинирования ранее вычисленных оптимальных значений. Динамическое программирование требует значительных вычислительных ресурсов, тогда как жадные алгоритмы, как правило, имеют меньшую временную сложность и требуют меньше памяти. Жадные алгоритмы применяются в самых разных областях, включая теорию графов, комбинаторную оптимизацию и задачи планирования. Например, в алгоритме Дейкстры для нахождения кратчайшего пути жадный выбор заключается в том, чтобы на каждом шаге выбирать вершину с минимальным расстоянием от начальной вершины. В алгоритмах построения минимального

остовного дерева, таких как алгоритмы Крускала и Прима, жадный подход заключается в том, чтобы последовательно добавлять рёбра с минимальным весом, не создавая циклов.

Алгоритм Хаффмана

Обычно для хранения данных и передачи сообщений используются коды фиксированной длины, например, код ASCII.

Множество символов представляются некоторым количеством кодовых слов равной длины, которая для кода ASCII равна восьми битам (1 байт). При этом для всех сообщений с одинаковым количеством символов требуется одинаковое количество битов при хранении и одинаковая ширина полосы пропускания при передаче.

Конечно, если сообщение написано, скажем, на английском языке, то вероятность появления в нем букв "z" намного меньше, чем вероятность появления букв "e". Это означает, что если для представления буквы "e" использовать более короткое кодовое слово, чем для представления буквы "z", то можно ожидать, что в среднем для хранения сообщения потребуется меньше памяти, а для его передачи - меньшая ширина канала.

В коде ASCII сообщение "easily" кодируется следующим образом:

01100101	01100001	01110011	01101001	01101100	01111001
e	a	s	i	l	y

для чего требуется 48 бит, в то время как при использовании кода со следующим представлением символов

1001	0	1010	11001	11010	1011
a	e	i	l	s	y

то же самое сообщение можно закодировать следующим образом

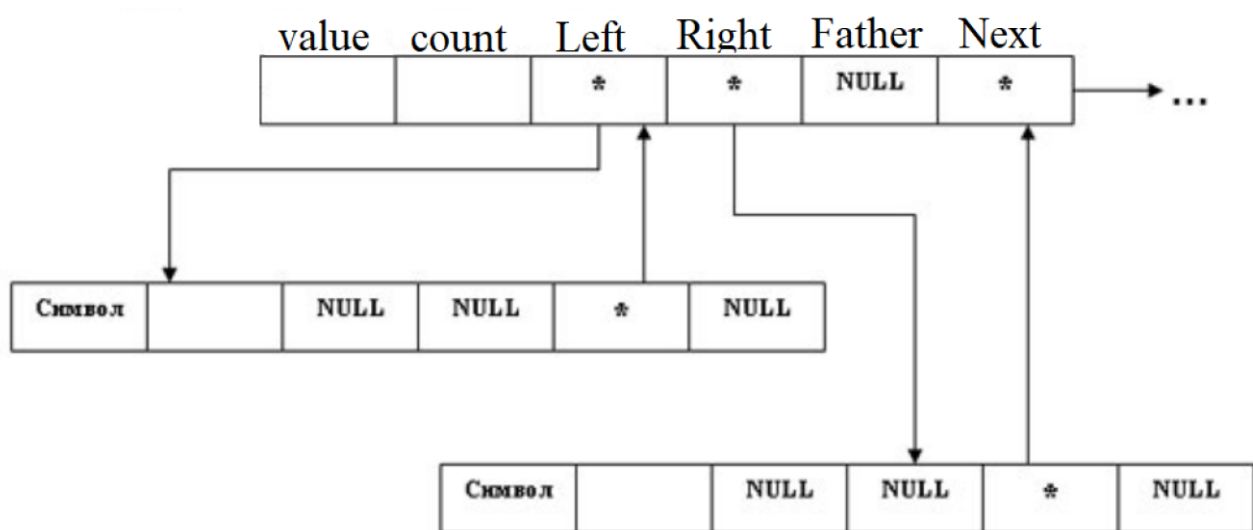
0	1	0	0	1	1	1	0	1	0	1	0	1	1	0	0	1	1	0	1	1
e	a			s				i				l				y				

используя только 23 бита.

Кодовые слова должны быть выбраны так, чтобы никакое из них не было префиксом любого другого кодового слова. Благодаря этому условию гарантируется возможность однозначного декодирования определённого закодированного текста.

В классической статье, опубликованной в 1952 г. Дэвид Хаффман описал алгоритм поиска множества кодов, которые минимизируют ожидаемую длину сообщений при условии, что известны вероятности появления каждого символа. Существенно, что в этом методе при определении длины кодовых слов символам, имеющим меньшую вероятность появления, ставятся в соответствие более длинные кодовые слова. После этого остается образовать некоторый однозначно декодируемый код с кодовыми словами надлежащей длины.

Хаффман в заключительном разделе работы отождествляет однозначное множество кодовых слов с двоичным деревом. Каждый лист дерева соответствует одному из символов. Глубина этого листа, т.е. его расстояние от корня, - это длина кодового слова соответствующего символа.



Цифры кодового слова являются адресом этого листа, т.е. последовательностью инструкций для продвижения от корня к листу, например, команда "0" - двигаться влево, а "1" - двигаться вправо. Тогда каждой вершине дерева будет приписано двоичное слово, описывающее, как добраться к этой вершине от корня. Самому корню соответствует пустое слово "0".

Хаффман пишет: "Так как объединение сообщений в составные сообщения подобно слиянию струек, ручейков и речушек в одну большую реку, описанную выше процедуру можно рассматривать по аналогии с расстановкой знаков жуком-плавунцом в каждом месте впадения притоков по пути его перемещения вниз по течению ... искомым будет код, который должен помнить плавунец, чтобы совершить обратный путь против течения".

Алгоритм Хаффмана:

1 этап:

Множество символов располагается в порядке уменьшения вероятностей их появления. Каждому из символов будет соответствовать лист дерева, следовательно, можно представить себе этот этап процесса как построение линейного списка, содержащего листья будущего дерева.

2 этап:

Производится повторяющееся сокращение числа максимальных непересекающихся поддеревьев посредством объединения двух "легчайших" деревьев для получения нового составного дерева.

Листья любого бинарного дерева образуют префиксный код, и, наоборот, для всякого префиксного кода существует такое дерево, что слова кода соответствуют его листьям.

Задание 4 (2 балла)

Задача:

Компания разрабатывает систему сжатия данных для передачи текстовой информации. Вам необходимо реализовать алгоритм Хаффмана, чтобы минимизировать количество бит, используемых для представления текста.

Реализовать следующие методы:

- а) Метод построения дерева;
- б) Метод построения списка частот символов;
- в) Метод шифрования (сжатия);
- г) Метод дешифровки;
- д) подсчет коэффициента сжатия.

Пример работы программы представлен ниже

```
Введите текст, содержащий не менее двух символов...
мама мыла раму
Полученный список:
м (0.285714) --> а (0.285714) --> (0.142857) --> ы (0.0714286) --> л (0.0714286) --> р (0.0714286) --> у (0.0714286) -->
Построим дерево...
      л (0.0714286)
      * (0.142857)
      * (0.428571)
      * (0.285714)
      * (1)
      * (0.285714)
      * (0.571429)
      * (0.142857)
      * (0.285714)
      * (0.0714286)
      * (0.142857)
      * (0.0714286)
-----
Приступим к кодировке введенного текста...
Код перед Вами... 100110011101000100001110111101101110
Коэффициент сжатия: 32.1429%
Ранее было зашифровано... мама мыла раму
Расшифровано...мама мыла раму
```

Решение:



Ответ:	
Задание 5*	
Задача:	Вы работаете в компании, разрабатывающей систему оптимизированной передачи текстовой информации по каналам связи с ограниченной пропускной способностью. Ваша задача -не просто закодировать текст с помощью алгоритма Хаффмана, но и минимизировать размер передаваемых данных, включая сам словарь кодировки. Для этого требуется: 1. Закодировать строку S с помощью алгоритма Хаффмана. 2. Минимизировать размер передаваемого сообщения, учитывая необходимость передачи словаря кодировки. 3. Оценить, можно ли сэкономить место за счёт битового хранения словаря.
Решение:	
Ответ:	

3. Примеры на жадные алгоритмы. Задача о размене монет

Теоретический материал
Некоторые классические задачи, решаемые жадными алгоритмами: 1. Задача о размене монет - выбор минимального количества монет для получения нужной суммы. 2. Задача о рюкзаке (фракционная версия) - выбор предметов с максимальным удельным весом (ценность/вес). 3. Задача о покрытии отрезков - выбор минимального количества интервалов для покрытия множества точек. 4. Алгоритмы минимального остовного дерева: - Алгоритм Крускала - добавляет рёбра в порядке возрастания веса. - Алгоритм Прима - строит остовное дерево, начиная с одной вершины.

5. Алгоритм Дейкстры - поиск кратчайшего пути в графе с неотрицательными весами рёбер.

Задача о размене монет

Условие:

Дано некоторое количество монет различных номиналов и сумма, которую необходимо разменять. Нужно найти минимальное количество монет, необходимое для размена данной суммы, используя жадный алгоритм.

Жадный подход:

1. Отсортируем монеты по убыванию номинала.
2. Начнём размен с монеты наибольшего номинала.
3. На каждом шаге выбираем максимальное количество монет данного номинала, не превышая сумму.
4. Переходим к монете меньшего номинала и повторяем процесс, пока сумма не будет разменяна.

Ограничение:

Жадный алгоритм работает корректно, если система монет обладает свойством жадного выбора, т.е. любой размен можно представить как сумму наибольших возможных номиналов (например, для стандартных монет 1, 5, 10, 25, 50 он работает, но для произвольных наборов может давать неоптимальный результат).

Пример 2

Задача:



Реализовать задачу о размене монет

Решение:

```

2 #include <iostream>
3 #include <vector>
4 #include <algorithm>
5
6 using namespace std;
7
8 // Функция для размена суммы минимальным числом монет
9 void minCoins(vector<int> &coins, int amount) {
10     // Сортируем номиналы монет по убыванию
11     sort(coins.rbegin(), coins.rend());
12
13     vector<int> count(coins.size(), 0); // Количество монет каждого номинала
14     int totalCoins = 0;
15
16     cout << "Размен для суммы: " << amount << endl;
17
18     for (size_t i = 0; i < coins.size(); i++) {
19         if (amount >= coins[i]) {
20             count[i] = amount / coins[i]; // Берем максимально возможное количество этой монеты
21             amount -= count[i] * coins[i]; // Обновляем оставшуюся сумму
22             totalCoins += count[i]; // Считаем общее количество монет
23         }
24     }
25
26     // Вывод результатов
27     if (amount == 0) {
28         cout << "Минимальное количество монет: " << totalCoins << endl;
29         cout << "Размен: " << endl;
30         for (size_t i = 0; i < coins.size(); i++) {
31             if (count[i] > 0) {
32                 cout << "Монета " << coins[i] << ": " << count[i] << " шт." << endl;
33             }
34         }
35     } else {
36         cout << "Размен невозможен с данным набором монет." << endl;
37     }
38 }
39
40 // Главная функция
41 int main() {
42     vector<int> coins = {1, 5, 10, 25, 50}; // Доступные номиналы монет
43     int amount;
44     cout << "Введите сумму для размена: ";
45     cin >> amount;
46
47     minCoins(coins, amount);
48
49     return 0;
50 }

```

Ответ:

```

Введите сумму для размена:
256
Размен для суммы: 256
Минимальное количество монет: 7
Размен:
Монета 50: 5 шт.
Монета 5: 1 шт.
Монета 1: 1 шт.

```

Задание 6 (2 балла)

Задача:

Дано N задач, каждая из которых имеет время начала и окончания. Нужно выбрать максимальное количество непересекающихся задач, чтобы выполнить их в одном рабочем дне. Сортируйте задачи по времени окончания и выбирайте те, которые начинаются после завершения предыдущей.

	Решение:
	Ответ:
	Задание 7*
	Задача:
	Есть N заказов, которые нужно доставить, и K курьеров. Каждый курьер может взять несколько заказов, но его общий маршрут не должен превышать T километров. Требуется минимизировать количество задействованных курьеров. Сортируйте заказы по дальности и жадно распределяйте их между курьерами.
	Решение:
	Ответ: